

SQL Injection Attack

A Comprehensive Study on Web Security's Most Persistent Threat

Cyber Security | IIT Jodhpur | Team 10

Group Members



Khushi Bawistale

G25AIT2052



Shashank Jangid

G25AIT2100



Rajath S M

G25AIT2079



Siddhant Singh

G25AIT2105



Deepanshu Arora

G25AIT2029



Soumya A

G25AIT2111



Shah Tirth R.

G25AIT2121

Agenda

- 01 Introduction to SQL Injection
- 02 How SQL Injection Works
- 03 Types of SQL Injection
- 04 Real-World Breaches & Impact
- 05 Attack Methodology Deep Dive
- 06 How ' OR 1=1 -- Works Internally
- 07 Breaking Down Each Character's Role
- 08 How the Database Evaluates the Query
- 09 Countermeasures & Defense Strategies
- 10 Secure Coding Best Practices
- 11 Key Takeaways & References

What is SQL Injection?



Definition

SQL Injection (SQLi) is a code injection technique that exploits vulnerabilities in an application's database layer. Attackers insert malicious SQL statements into input fields, tricking the application into executing unintended commands on the database.

26%

of all data breaches
in 2024

2,600

SQLi CVEs
in 2025

+

How the Attack Flows



Attacker

Crafts malicious
SQL payload



Web App

Passes unsanitized
input to query



Database

Executes the
injected SQL



Data Leaked

Sensitive data
exfiltrated

How SQL Injection Works



Normal Login Query

User input: admin

```
SELECT * FROM users
```

```
WHERE name = 'admin'
```



Injected Malicious Query

User input: ' OR '1'='1' --

```
SELECT * FROM users
```

```
WHERE name='' OR '1'='1' --'
```

Why Does This Work?

The application concatenates user input directly into the SQL query string without sanitization.

The injected ' OR '1'='1' condition always evaluates to TRUE, returning all rows.

The double dash -- comments out the remaining SQL, preventing syntax errors.

Types of SQL Injection



In-Band SQLi

Most common type. Results are visible directly in the web response.

Error-Based

Forces DB errors to leak schema info such as table and column names

Union-Based

Uses UNION operator to combine malicious results with legitimate query output



Blind SQLi

No visible output. Data inferred from app behavior or response timing.

Boolean-Based

Sends true/false conditions and infers data from application behavior changes

Time-Based

Uses SLEEP / WAITFOR to detect conditions via response time differences



Out-of-Band SQLi

Data exfiltrated via a separate channel such as DNS or HTTP.

DNS Exfiltration

DB initiates DNS lookup to attacker-controlled server carrying data

HTTP Channel

DB sends HTTP request with extracted data to external endpoint

Real-World Breaches & Impact

2023

MOVEit File Transfer Breach

Mass SQLi exploitation in a managed file transfer application. Thousands of organizations impacted globally, stolen data sold on dark web.

2024

BeyondTrust + PostgreSQL

Zero-day SQLi in BeyondTrust PRA/RS products combined with PostgreSQL vulnerability (CVE-2025-1094). Targeted US government agencies.

2025

Major Retail Chain Breach

Attackers exploited SQL injection to extract credit card data and personal details of millions of customers from a major retailer.

2025

GenAI-Generated Code Vulnerability

A startup deployed AI-generated code with unvalidated SQL queries. 3,200 user records exposed within 48 hours of launch.

Attack Methodology Deep Dive

01



Reconnaissance

Identify input fields that interact with the database (login forms, search bars, URL parameters, cookies).

02



Testing & Detection

Inject special characters (single quotes, semicolons) and observe error messages or behavioral changes.

03



Exploitation

Craft payloads to extract data, bypass authentication, or manipulate database records using UNION, subqueries, or stacked queries.

04



Privilege Escalation

Attempt to read system tables, access admin credentials, or execute OS commands via database functions like xp_cmdshell.

05



Data Exfiltration

Extract sensitive data (credentials, PII, financial records) through in-band responses, error messages, or out-of-band channels.

Vulnerable vs. Secure Code



Vulnerable Code (PHP) - Direct String Concatenation

```
$username = $_GET['user']; // Unsanitized input  
  
$query = "SELECT * FROM users WHERE name='" . $username . "'";  
  
$result = mysqli_query($conn, $query);
```



Secure Code (PHP) - Parameterized Query / Prepared Statement

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE name = ?");  
  
$stmt->execute([$_GET['user']]); // Input treated as data, not code  
  
$result = $stmt->fetchAll();
```



Prepared statements separate SQL logic from user data, making injection structurally impossible.

How ' OR 1=1 -- Works Internally

1 You type the payload in the login form

ROLL NUMBER

' OR 1=1 --

PASSWORD

anything

The browser sends these values to the Flask server via HTTP POST. No validation, no sanitization.

2 Server builds the SQL query via string concatenation

```
query = f"SELECT * FROM students WHERE roll_number = '{username}' AND password = '{password}'"
```

After substitution, the actual SQL becomes:

```
SELECT * FROM students WHERE roll_number = '' OR 1=1 --' AND password = 'anything'
```

KEY Your input breaks out of the string literal and becomes executable SQL code.

Breaking Down Each Character's Role

Each part of the payload ' OR 1=1 -- serves a specific purpose in the attack:

Character(s)	What It Does
'	Closes the opening quote of roll_number = ' — escapes from string context into SQL context
OR	Adds a new SQL condition — the database evaluates: (original condition) OR (your condition)
1=1	A condition that is always TRUE — since 1 always equals 1, every row in the table matches
--	SQL comment — everything after this is ignored, killing the AND password = '...' check

Payload breakdown: ' close quote **OR** add condition **1=1** always true -- comment out rest

How the Database Evaluates the Query

```
WHERE roll_number = '' OR 1=1
```

The database checks each row against this logic. For every single row:

Condition	Result	Why
roll_number = ''	FALSE	No student has an empty roll number
1 = 1	TRUE	1 always equals 1
FALSE OR TRUE	TRUE	OR only needs one side to be true

RESULT Since 1=1 is TRUE for every row, the WHERE clause matches all 10 students in the table.

! The app treats the first row as a successful login

Attack successful: You are logged in as the first student without knowing any password. The dashboard also displays all 10 students' data including their passwords.

Countermeasures: Primary Defenses



Parameterized Queries

PRIMARY

Use prepared statements that separate SQL logic from user-supplied data. This is the #1 defense recommended by OWASP. User input is always treated as data, never as executable code.



Input Validation & Whitelisting

PRIMARY

Validate all user inputs against expected patterns. Use whitelists for acceptable characters and formats. Enforce strict type checks and length limits on every input field.



ORM Frameworks

PRIMARY

Use Object-Relational Mappers such as SQLAlchemy, Hibernate, or Django ORM. These enforce parameterized queries by default, significantly reducing the attack surface.

Countermeasures: Secondary Defenses



Least Privilege Principle

SECONDARY

Database accounts used by applications should have minimal permissions. Avoid granting DROP, ALTER, or administrative access to application-level accounts.



Web Application Firewall (WAF)

SECONDARY

WAFs inspect HTTP traffic and block requests containing SQLi attack patterns. Useful as an additional layer for legacy applications, but not a standalone solution.



Regular Security Testing

SECONDARY

Conduct SAST/DAST scans, penetration testing, and code reviews regularly. Use tools like SQLMap, Burp Suite, and OWASP ZAP to identify vulnerabilities before attackers do.

Secure Coding Best Practices

Never trust user input

Treat all input (form fields, URL parameters, cookies, HTTP headers) as potentially malicious. Validate and sanitize everything.

Implement proper error handling

Never expose database error messages to end users. Use generic error pages and log detailed errors server-side only.

Use parameterized queries everywhere

Every database interaction must use prepared statements or parameterized queries, with no exceptions for "simple" queries.

Keep frameworks and libraries updated

Regularly update your tech stack. Many SQLi vulnerabilities exist in outdated framework versions and third-party libraries.

Apply the principle of least privilege

Each application should connect to the database with the minimum permissions required for its functionality.

Audit AI-generated code

Code generated by AI tools may contain SQLi vulnerabilities. Always review and test generated code before deployment.

Key Takeaways



SQL Injection is over 25 years old yet remains in the OWASP Top 10 and caused 26% of breaches in 2024.



A single vulnerable input field can compromise an entire database, leading to data theft, manipulation, or destruction.



Parameterized queries are the most effective defense and should be the standard for all database interactions.



Defense in depth is essential: combine prepared statements, input validation, WAFs, least privilege, and security testing.



AI-generated code is introducing new SQLi risks. Always audit and test code regardless of its source.

Group 10 Team Members Contribution



Khushi Bawistale G25AIT2052

Research, Presentation and demonstration of in-band SQLi injection



Soumya A G25AIT2111

Research and Presentation



Shashank Jangid G25AIT2100

Research, Demonstration of Inband SQLi injection in insecure and secure bank and Presentation



Rajath S M G25AIT2079

Research, Demonstration of Blind, UNION, OOB SQLi injection and Presentation



Siddhant Singh G25AIT2105

Research, Presentation and demonstration of In-band SQLi injection in IITJ simulated portal



Deepanshu Arora G25AIT2029

Research, Presentation and demo discussion



Shah Tirth R. G25AIT2121

Research, Presentation and hands on experience discussion

Thank You!

| Questions are welcome — Cyber Security | IIT Jodhpur - Team 10